

**ANALISIS PROYEK
MATA KULIAH KECERDASAN BUATAN**

**IMPLEMENTASI ALGORITMA A* (A-STAR) DAN DIJKSTRA
BERBASIS OSRM PADA SISTEM NAVIGASI RUTE KAMPUS
UNIVERSITAS BENGKULU (UNIB Route Navigation System)**



Disusun Oleh :

R. Noor Fikhri

G1A024075

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

2026

1. ANALISIS MEKANISME REDUKSI GRAF (OSRM)

1.1 Representasi Graf Kawasan Kampus UNIB

Sebelum membahas mekanisme reduksi, penting untuk menetapkan representasi matematis dari domain permasalahan. Jaringan jalan kawasan Universitas Bengkulu, sebagaimana dimodelkan oleh OpenStreetMap dan diproses oleh OSRM, dapat direpresentasikan sebagai graf berbobot berarah $G = (V, E, w)$, di mana V adalah himpunan simpul (persimpangan dan titik jalan), E adalah himpunan sisi berarah (segmen jalan), dan $w: E \rightarrow \mathbb{R}^+$ adalah fungsi bobot yang memetakan setiap sisi ke nilai non-negatif berupa waktu tempuh atau jarak. Dalam konteks kampus UNIB, skala graf yang dihadapi OSRM tidak terbatas pada 19 titik manual yang didefinisikan dalam sistem—titik-titik tersebut hanyalah titik acuan (waypoints) di atas lapisan graf OSM yang sesungguhnya dapat terdiri dari ribuan simpul dan puluhan ribu sisi yang merepresentasikan setiap potongan trotoar, jalan kampus, dan persimpangan kecil. Permasalahan inilah yang menjadikan Dijkstra atau A* murni di sisi klien tidak praktis.

1.2 Contraction Hierarchies (CH): Reduksi Topologis

1.2.1 Prinsip Matematika CH

Contraction Hierarchies (CH) adalah teknik pra-proses graf yang diperkenalkan oleh Geisberger et al. (2008). Inti dari CH adalah operasi node contraction: secara iteratif, simpul v dengan bobot kepentingan terendah dieliminasi dari graf aktif, dan untuk setiap pasang tetangga (u, w) melalui v , sebuah shortcut edge $(u \rightarrow w)$ ditambahkan jika dan hanya jika jalur $u \rightarrow v \rightarrow w$ merupakan jalur terpendek satu-satunya dari u ke w (kondisi ini disebut witness search).

Secara formal, sebuah shortcut (u, w) dengan bobot $w(u,v) + w(v,w)$ ditambahkan jika tidak terdapat path alternatif P dari u ke w (tanpa melalui v) dengan total bobot $\leq w(u,v) + w(v,w)$. Proses ini menghasilkan augmented graph $G^* = (V, E \cup E_{\text{shortcuts}}, w)$ yang secara topologis lebih padat tetapi memiliki struktur hierarkis. Simpul-simpul diberi level prioritas: simpul yang dikontrak

lebih dulu memiliki level lebih rendah, sedangkan simpul yang bertahan lebih lama dalam proses kontraksi (umumnya persimpangan besar atau jalan arteri) memiliki level tinggi.

Setelah pra-proses CH, setiap jalur terpendek dari titik s ke titik t dapat didekomposisi sebagai gabungan dari upward path (jalur dari s ke atas hierarki) dan downward path (turun dari hierarki ke t). Ini memungkinkan algoritma kueri berjalan bidireksional secara eksklusif pada simpul berprioritas tinggi, yang hanya merupakan fraksi kecil dari total simpul V .

1.2.2 Kompleksitas dan Perbandingan dengan Dijkstra Murni

Misalkan kampus UNIB dalam konteks OSM memiliki $|V|$ simpul dan $|E|$ sisi. Dijkstra murni pada graf ini akan mengeksekusi ekspansi dengan kompleksitas $O(|E| + |V| \log |V|)$ dan menjelajahi simpul ke segala arah secara merata, tanpa panduan menuju tujuan. Untuk jaringan jalan yang representatif dengan $|V| \approx 10.000$ dan $|E| \approx 30.000$, Dijkstra murni di klien akan memerlukan ratusan ribu operasi per kueri.

Sebaliknya, CH memotong ruang pencarian secara dramatis. Riset empiris Geisberger et al. menunjukkan bahwa pada jaringan jalan Eropa Barat ($|V| \sim 18$ juta), rata-rata hanya ~ 1.000 simpul yang dieksplorasi per kueri CH, dibandingkan dengan jutaan simpul jika menggunakan Dijkstra penuh. Waktu kueri turun dari detik (Dijkstra) menjadi sub-milidetik (CH). Properti reduksi ini berlaku secara proporsional pada skala kampus.

$$T_{\text{query}}(\text{CH}) = O(|V_{\text{contracted}}|) \ll O(|E| + |V| \log |V|) = T_{\text{query}}(\text{Dijkstra})$$

Perlu ditegaskan bahwa biaya reduksi ini dibayar pada fase pra-proses: konstruksi CH memerlukan $O(|V| \log |V|)$ waktu dan memori yang signifikan proses yang tidak dapat dijalankan di sisi klien secara real-time. Inilah alasan fundamental mengapa OSRM wajib beroperasi sebagai server-side engine.

1.3 Multilevel Dijkstra (MLD): Penggabungan Strategi Hierarkis

Multilevel Dijkstra (MLD), atau yang dalam implementasi OSRM dikenal sebagai Multi-Level Dijkstra dengan customizable partitions (CRP/MLC), membangun hierarki di atas pembagian graf (graph partitioning) ketimbang kontraksi simpul tunggal. Pada MLD, graf G dibagi menjadi k level, di mana setiap level merupakan rangkuman (overlay) dari level sebelumnya. Setiap overlay hanya mempertahankan sisi-sisi boundary yang menghubungkan partisi berbeda.

Saat mengeksekusi kueri rute dari s ke t , algoritma MLD secara efisien memilih level abstraksi yang sesuai: untuk segmen rute yang jauh dari s dan t , algoritma naik ke level tinggi (yang memiliki lebih sedikit simpul tetapi merangkum koneksi antar partisi); untuk segmen dekat s dan t , algoritma turun ke level rendah untuk mendapatkan rute lokal yang presisi. Strategi ini menghasilkan jalur yang secara matematis sama dengan Dijkstra penuh, tetapi dengan jumlah operasi yang jauh lebih kecil.

Dalam implementasi OSRM yang digunakan oleh UNIB Route Navigation System, kedua teknik ini digabungkan: pra-proses offline menggunakan CH atau MLD tergantung konfigurasi server, dan kueri dijalankan secara bidireksional. Sistem navigasi UNIB hanya perlu mengirimkan koordinat s (asal) dan t (tujuan) melalui protokol OSRM v1 API, dan menerima kembali respons berupa array koordinat polyline dalam format JSON—tanpa perlu mengetahui satu pun detail internal operasi CH atau MLD tersebut.

2. ANALISIS PERHITUNGAN BOBOT

STUDI KASUS RUTE FEB → FAKULTAS TEKNIK

2.1 Parameter Studi Kasus

Studi kasus yang dianalisis menggunakan data terukur aktual dari pengujian sistem. Rute dieksekusi antara dua titik yang terletak dalam kawasan Universitas Bengkulu dengan parameter sebagai berikut:

Titik Asal (s): Fakultas Ekonomi dan Bisnis (FEB) UNIB | Koordinat: (-3.7617, 102.2686) Titik Tujuan (t): Fakultas Teknik Universitas Bengkulu | Koordinat: (-3.7583, 102.2774) Jarak Optimal (d): 1,46 km = 1.460 meter (diperoleh dari OSRM routing engine) Hasil Mode Kendaraan (t_drive): 5 menit (diperoleh dari r.summary.totalTime OSRM) Hasil Mode Jalan Kaki (t_walk): 17 menit (dihitung oleh logika konversi klien)

2.2 Mekanisme Kalkulasi Bobot Graf oleh OSRM (Mode Kendaraan)

Pada mode kendaraan, OSRM tidak sekadar menghitung jarak geometris (Euclidean distance) antara dua koordinat. Fungsi bobot $w(e)$ untuk setiap sisi e dalam graf OSM dimodelkan sebagai fungsi waktu tempuh:

$$w(e) = \text{length}(e) / \text{speed_profile}(e, \text{road_type}, \text{access_rules})$$

di mana speed_profile adalah fungsi yang memetakan tipe jalan (primary, secondary, residential, service, dll.) dan aturan akses (kendaraan bermotor tidak dapat melewati jalur pejalan kaki) ke nilai kecepatan tempuh yang sesuai. Ini berarti dua segmen jalan dengan panjang fisik yang sama dapat memiliki bobot yang berbeda tergantung tipe jalannya.

Hasil rute dari FEB ke Fakultas Teknik sepanjang 1,46 km dengan waktu tempuh 5 menit menyiratkan rata-rata kecepatan efektif kendaraan:

$$v_{\text{eff}} = d / t = 1.460 \text{ m} / (5 \times 60 \text{ s}) = 1.460 / 300 \approx 4,87 \text{ m/s} \approx 17,5 \text{ km/jam}$$

Nilai ini konsisten dengan kecepatan kendaraan di dalam kawasan kampus yang dipenuhi pejalan kaki, dengan adanya belokan, hambatan, dan batas kecepatan internal. OSRM memperoleh $t_{\text{drive}} = 5$ menit langsung dari nilai $r.\text{summary.totalTime}$ yang merupakan akumulasi bobot $w(e)$ sepanjang jalur optimal $P^* = \text{argmin}_P \sum w(e)$ untuk $e \in P$.

2.3 Mekanisme Konversi Waktu Jalan Kaki (Kalkulasi Sisi Klien)

Terdapat perbedaan komputasi yang fundamental antara mode kendaraan dan mode jalan kaki dalam sistem ini. Untuk mode jalan kaki, profil OSRM (foot profile) memang secara internal mempertimbangkan jalur pedestrian (trotoar, area pejalan kaki) yang tidak dapat dilalui kendaraan. Namun, nilai $r.\text{summary.totalTime}$ dari OSRM untuk profil jalan kaki tidak selalu tersedia secara konsisten di semua versi server publik.

Oleh karena itu, logika sisi klien (client-side logic) mengimplementasikan konversi waktu berdasarkan konstanta biometrik rata-rata kecepatan jalan kaki manusia sebesar 1,4 m/s (setara dengan ± 5 km/jam). Secara matematis:

$$t_{\text{walk}} = d / v_{\text{walk}} = 1.460 \text{ m} / 1,4 \text{ m} \cdot \text{s}^{-1} = 1.042,86 \text{ s} \approx 1.043 \text{ s}$$
$$t_{\text{walk}} (\text{dalam menit}) = \text{floor}(1.043 / 60) = \text{floor}(17,38) = 17 \text{ menit}$$

Fungsi `Math.floor()` dipilih secara sengaja untuk menghasilkan estimasi waktu yang konservatif (membulatkan ke bawah) ketimbang ceiling, sehingga estimasi yang ditampilkan kepada pengguna tidak mengecewakan karena terlalu pesimistis.

2.4 Analisis Deviasi: Mengapa Jarak Identik Menghasilkan Waktu Berbeda?

Untuk rute spesifik ini, jarak yang dihasilkan kedua mode adalah identik (1,46 km). Ini berarti OSRM driving profile dan foot profile menghasilkan polyline yang secara geometris sama, karena tidak ada jalur khusus pejalan kaki yang merupakan shortcut signifikan antara kedua titik tersebut. Namun, disparitas waktu yang sangat signifikan—faktor 3,4 kali lebih lama untuk jalan kaki

sepenuhnya disebabkan oleh perbedaan fungsi bobot yang diterapkan pada jalur yang sama:

Parameter	Mode Kendaraan	Mode Jalan Kaki	Rasio / Keterangan
Jarak Tempuh (d)	1,46 km	1,46 km	Identik — jalur geometris sama
Kecepatan Efektif (v)	$\approx 17,5$ km/jam (4,87 m/s)	5,04 km/jam (1,4 m/s)	Rasio: 3,48× lebih cepat kendaraan
Sumber Nilai Waktu	r.summary.totalTime (OSRM server)	d / 1,4 m/s (kalkulasi klien)	Dua sumber berbeda; satu server, satu klien
Estimasi Waktu Akhir	5 menit	17 menit	Selisih 12 menit; rasio waktu 3,4×

Implikasi algoritmik dari perbedaan ini adalah bahwa fungsi bobot graf $w(e)$ merupakan dimensi komputasi yang tidak kalah pentingnya dari topologi graf itu sendiri. Dua algoritma pencarian jalur yang identik secara topologis dapat menghasilkan waktu estimasi yang sangat berbeda semata-mata karena perbedaan dalam definisi fungsi bobot yang digunakan.

3. TABEL KOMPARASI ALGORITMA

Tabel 1 berikut menyajikan perbandingan formal dan komprehensif antara tiga entitas algoritmik yang dianalisis, mencakup seluruh dimensi komputasi yang relevan bagi sistem navigasi berbasis Web GIS.

Tabel 1. Perbandingan Komparatif: Dijkstra Konvensional vs. A* Standar vs. OSRM (MLD + CH)

Metrik Komparasi	Dijkstra Konvensional	A* (A-Star) Standar	OSRM (MLD + CH)
Kompleksitas Waktu (Worst Case)	$O(E + V \log V)$ dengan Fibonacci Heap	$O(b^d)$ — bergantung heuristik; $O(E \log V)$ dalam praktik	$O(V)$ kueri setelah pra-proses; pra-proses $O(V \log V)$
Kompleksitas Waktu (Praktis / Dunia Nyata)	Lambat pada graf besar; menjelajahi semua arah	Lebih cepat dari Dijkstra; bergantung pada kualitas $h(n)$	Sub-milidetik; telah dievaluasi pada peta sekelas benua
Kompleksitas Ruang (Memori)	$O(V + E)$ — menyimpan semua simpul dan sisi	$O(b^d)$ — open/closed list dapat membengkak; $O(V)$ dalam kasus terbaik	Tinggi di pra-proses (CH shortcut graph); $O(\log V)$ saat kueri
Struktur Data Inti	Min-Priority Queue (Min-Heap); Distance Array	Min-Heap berdasarkan $f(n)=g(n)+h(n)$; Open & Closed Set	Hierarchical Shortcut Graph; Overlay Graph dengan CH Levels; Bucket Heaps

Metrik Komparasi	Dijkstra Konvensional	A* (A-Star) Standar	OSRM (MLD + CH)
Overhead per Iterasi	Relaksasi sisi: mengecek semua tetangga tanpa panduan arah	Kalkulasi $h(n)$ per simpul (Haversine/Euclidean) ; sedikit overhead dibanding Dijkstra	Hampir nol saat kueri; overhead besar tersembunyi di fase pra-proses offline
Kepastian Optimalitas Jarak	TERJAMIN — optimal selama bobot sisi non-negatif	TERJAMIN — jika $h(n)$ bersifat admissible ($h(n) \leq h^*(n)$)	TERJAMIN — CH mempertahankan optimalitas; MLD menggunakan level-query dengan jaminan korektansi
Arah Ekspansi Graf	Omnidireksional (semua arah secara merata)	Terarah menuju tujuan (guided by $h(n)$)	Bidireksional pada CH; ekspansi hanya pada simpul 'penting' di hirarki tinggi
Ketergantungan Pra-Proses	Tidak ada — langsung pada graf mentah	Tidak ada — langsung; $h(n)$ dihitung on-the-fly	WAJIB — CH + MLD memerlukan pra-proses offline data OSM berjam-jam

Metrik Komparasi	Dijkstra Konvensional	A* (A-Star) Standar	OSRM (MLD + CH)
Kesesuaian Arsitektur	Cocok client- side untuk graf kecil ($< \sim 500$ simpul)	Cocok client-side untuk navigasi lokal dengan ruang terbatas	HARUS server- side; tidak layak di browser karena beban memori pra- proses

Catatan interpretasi Tabel 1: Kompleksitas OSRM yang terlihat lebih buruk pada fase pra-proses adalah trade-off yang disengaja: pekerjaan berat dilakukan sekali secara offline di server, sehingga setiap kueri real-time dari pengguna mendapatkan manfaat dari reduksi tersebut. Dari perspektif pengguna sistem UNIB Route Navigation System, kompleksitas yang relevan adalah kompleksitas kueri (bukan pra-proses), yaitu $O(|V|)$ yang sub-milidetik.

4. MATRIKS KESIMPULAN KOMPUTASI DAN REKOMENDASI

Tabel 2 menyajikan matriks rekomendasi berdasarkan kondisi graf dan skenario arsitektur. Setiap baris merupakan sebuah kondisi atau skenario penggunaan yang berbeda, dan kolom terakhir menyajikan konklusi rasional mengenai pilihan algoritma yang paling optimal.

Tabel 2. Matriks Rekomendasi: Kondisi Graf vs. Pemilihan Algoritma Optimal

Kondisi / Skenario Graf	Dijkstra	A* Standar	OSRM (MLD+CH)	Rekomendasi & Rasionalisasi
Graf kecil (< 300 simpul); semua pasang titik perlu dihitung	OPTIMAL	Dapat digunakan	Tidak perlu	Dijkstra; overhead pra-proses OSRM tidak terjustifikasi
Navigasi satu titik ke satu tujuan; ruang pencarian terbatas dan diketahui	Boros	OPTIMAL	Dapat digunakan	A*; heuristik Haversine memberikan panduan langsung menuju tujuan
Navigasi real-time pada	Tidak praktis	Lambat; $h(n)$ kurang efektif di	OPTIMAL	OSRM; CH + MLD

Kondisi / Skenario Graf	Dijkstra	A* Standar	OSRM (MLD+CH)	Rekomendasi & Rasionalisasi
jaringan jalan skala kota/nasional (OSM data)		jalan berliku		memungkinkan sub-milidetik bahkan untuk benua penuh
Arsitektur murni client-side (browser); tidak ada server backend	Terlaksana untuk graf lokal kecil	Terlaksana untuk graf lokal kecil	TIDAK LAYAK tanpa API eksternal	Delegasikan ke API OSRM publik — strategi paling optimal: zero komputasi di klien
Ketersediaan jaringan internet terbatas (offline/intranet)	OPTIMAL — berjalan penuh di klien	OPTIMAL — berjalan penuh di klien	Memerlukan self-hosted OSRM server	A* dengan graf statis kampus — ringan, dapat dikodekan langsung di klien
Proyek	Tidak	Tidak	DIIMPLEMENTA	Strategi

Kondisi / Skenario Graf	Dijkstra	A* Standar	OSRM (MLD+CH)	Rekomendasi & Rasionalisasi
UNIB Route Navigation System (arsitektur aktual: full client-side + API OSRM publik)	diimplementasikan langsung	diimplementasikan langsung	SIKAN — via Leaflet Routing Machine + OSRM v1 API	OPTIMAL : beban komputasi CH + MLD sepenuhnya di server OSRM; klien hanya mem-render polyline

Konklusi Utama Matriks: Baris terakhir Tabel 2 mencerminkan kondisi aktual sistem UNIB Route Navigation System. Arsitektur client-side murni tanpa server backend menjadikan pendelegasian komputasi ke API OSRM publik sebagai satu-satunya strategi yang memungkinkan penggunaan kekuatan penuh algoritma MLD + CH. Browser klien hanya bertanggung jawab untuk mengirimkan dua koordinat dan merender polyline nol beban komputasi graf.

5. SINTESIS DAN PENUTUP

5.1 Sintesis Temuan Komputasi

Analisis yang disajikan dalam dokumen ini mengonfirmasi tiga temuan komputasi utama. Pertama, penggunaan OSRM sebagai backend routing engine secara matematis superior dibandingkan implementasi Dijkstra atau A* murni di sisi klien untuk jaringan jalan skala dunia nyata. Superioritas ini bukan disebabkan oleh perbedaan algoritma dasar semata, melainkan oleh teknik pra-proses Contraction Hierarchies dan Multilevel Dijkstra yang mentransformasi masalah pencarian jalur $O(|E| + |V| \log |V|)$ menjadi permasalahan kueri $O(|V_{\text{contracted}}|)$ yang sub-milidetik.

Kedua, studi kasus rute FEB ke Fakultas Teknik (1,46 km; 5 menit kendaraan; 17 menit jalan kaki) mendemonstrasikan bahwa fungsi bobot graf merupakan dimensi komputasi yang kritis dan independen dari topologi. Disparitas waktu sebesar faktor $3,4\times$ antara kedua mode perjalanan pada jarak geometris yang identik sepenuhnya berasal dari perbedaan definisi fungsi bobot $w(e)$ yang diterapkan pada jalur yang sama.

Ketiga, arsitektur pendelegasian yang diterapkan dalam sistem—di mana klien hanya berperan sebagai konsumen hasil komputasi server OSRM melalui REST API—merupakan strategi rekayasa perangkat lunak yang paling rasional untuk sistem navigasi berbasis Web GIS dengan kendala full client-side. Strategi ini memungkinkan pengguna menikmati kualitas algoritma kelas dunia (MLD + CH) tanpa memerlukan infrastruktur komputasi khusus di sisi pengguna.

5.2 Rekomendasi Pengembangan Berbasis Analisis

Berdasarkan analisis ini, dua rekomendasi berbasis pertimbangan algoritmik dapat diajukan untuk pengembangan lanjutan sistem. Rekomendasi pertama adalah mengimplementasikan self-hosted OSRM server di infrastruktur Universitas Bengkulu. Selain mengurangi latensi jaringan, pendekatan ini memungkinkan kustomisasi profil bobot (speed profile) sesuai karakteristik unik kawasan kampus—misalnya, mendefinisikan kecepatan kendaraan kampus yang

lebih rendah atau memperhitungkan bobot berbasis kepadatan mahasiswa pada jam-jam tertentu.

Rekomendasi kedua adalah mempertimbangkan implementasi A* lokal sebagai fallback mechanism untuk skenario offline atau konektivitas rendah. Mengingat graf kampus UNIB dapat dimodelkan dengan hanya beberapa ratus hingga beberapa ribu simpul (jauh di bawah skala OSM global), A* dengan heuristik Haversine distance akan memberikan kinerja yang memadai secara client-side—dan tetap menjamin optimalitas jalur selama fungsi heuristik terbukti admissible.